🔍 Basics of Prometheus – An Introduction to Monitoring & Observability 🛠️📈

If you're a DevOps engineer, SRE, or cloud-native enthusiast, understanding Prometheus is *non-negotiable* in 2025.

Here's a detailed breakdown on Prometheus, its use cases, architecture, configuration, and must-know queries.

🚀 What is Prometheus?

Prometheus is an **open-source systems monitoring and alerting toolkit**, originally built at SoundCloud and now a part of the **Cloud Native Computing Foundation (CNCF)**. It is purpose-built for **metrics-based monitoring** in modern microservices environments.

- Developed in **Go**
- Pull-based model
- Powerful **PromQL** querying language
- Time-series data storage
- Seamless **Grafana integration**

📌 Why Use Prometheus? — Use Cases

- ✅ **Infrastructure Monitoring** – VMs, containers, networks
- ✅ **Application Monitoring** – JVM, Go, Node.js, Python, etc.
- ✅ **Kubernetes Cluster Monitoring** – pods, nodes, services
- ✅ **Service-Level Objectives (SLOs)** – latency, error rate, etc.
- ✅ **Alerting System** – integrates with Alertmanager
- ✅ **Multi-dimensional Data Model** – metrics with labels

Prometheus is ideal for **cloud-native**, **distributed**, and **ephemeral** environments like Kubernetes.

Prometheus is not ephemeral — it's the **perfect tool to monitor ephemeral systems** like Kubernetes.

□ Prometheus Architecture – Key Components

1. **Prometheus Server**
    - Fetches metrics via HTTP (pull model)
    - Stores time-series data in its own TSDB
2. **Exporters**
    - Agents that expose metrics to Prometheus
    - e.g. Node Exporter, Blackbox Exporter, mysqld_exporter
3. **Push Gateway**
    - Accepts metrics from short-lived jobs (push model)
4. **Alertmanager**
    - Handles alerts (grouping, silencing, routing)
    - Integrates with Slack, Email, PagerDuty, Opsgenie
5. **Service Discovery**
    - Automatically finds targets (Kubernetes, EC2, Consul)
6. **PromQL**
    - Flexible query language for extracting and visualizing metrics

---

⚙️ Prometheus Simple Configuration File

- The main config file is: prometheus.yml

```
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: 'node'
    static_configs:
      - targets: ['localhost:9100']
  - job_name: 'kubernetes-nodes'
    kubernetes_sd_configs:
      - role: node
```

Key Sections:

🔹 global: Default settings like scrape_interval, evaluation_interval
🔹 scrape_configs: Define targets and jobs
🔹 relabel_configs: Modify discovered target labels
🔹 alerting: Configuration for Alertmanager
🔹 rule_files: Point to recording and alerting rules

🎯 What Are Targets in Prometheus?

In Prometheus, targets are the endpoints from which Prometheus collects metrics.

Each target exposes metrics over HTTP in a format that Prometheus understands. These endpoints are usually provided by exporters, applications, or services that emit metrics.

📌 Key Concepts:

🔷 Targets are defined in scrape_configs in prometheus.yml.

🔷 Each job can have one or more targets.

🔷 Prometheus "scrapes" these targets at a regular interval (default: 15s).

🔷 Targets can be statically defined or dynamically discovered (e.g. via Kubernetes, EC2, Consul).

🔗 Example: Static Target Configuration

```
scrape_configs:
  - job_name: 'node'
    static_configs:
      - targets: ['localhost:9100']
```

This means Prometheus will scrape metrics from the node_exporter running on localhost port 9100.

🔄 Dynamic Target Discovery (Kubernetes Example)

```
scrape_configs:
  - job_name: 'kubernetes-nodes'
    kubernetes_sd_configs:
      - role: node
```

Prometheus will automatically discover all Kubernetes nodes as targets.

📊 Prometheus Query Language (PromQL) – Basic queries for simple understanding. In real-time scenarios, the queries tend to be longer and more complex.

Query	Description
up	Check if targets are healthy (1 = up, 0 = down)
node_cpu_seconds_total	View total CPU seconds used
rate(http_requests_total[5m])	Requests per second
avg_over_time(node_memory_MemAvailable_bytes[10m])	Avg memory available over last 10m
topk(5, rate(http_requests_total[1m]))	Top 5 endpoints with most requests
sum by (instance)(rate(node_cpu_seconds_total[1m]))	CPU usage grouped by instance
node_filesystem_avail_bytes / node_filesystem_size_bytes	Disk space available as ratio
ALERTS{alertstate="firing"}	View firing alerts

📣 Prometheus + Grafana = 🔥

Prometheus integrates seamlessly with **Grafana**, enabling visually appealing dashboards for:

- CPU / memory / disk
- Kubernetes cluster health
- Application performance
- Custom SLO dashboards

💡 Pro Tips

✔ Use **recording rules** to precompute expensive queries
✔ Tune **scrape intervals** based on metric granularity needs
✔ Use **labels wisely** – too many can explode cardinality
✔ Secure endpoints with basic auth / TLS
✔ Integrate with **Alertmanager** for real-time notifications

---

💬 Final Thoughts

Prometheus is not just a monitoring tool — it's a **mindset** for building observability into modern systems. Whether you're running Kubernetes, EC2, or bare metal, Prometheus can be your single source of metrics truth.